UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Exposición

Tema 4.6. El Apocalipsis digital y la profecía del Y2K38

Grupo: 07

- López Granados Derek André
- Ortega Ayala Fernando

Semestre 2026-1

Fecha: 23/ abril / 2026

ÍNDICE

Introducción: El Apocalipsis Digital y la Profecía del Y2K38 1

1. La leyenda de John Titor: el viajero del tiempo 1

2. El origen: el problema del año 2000 (Y2K) 1-2

2.1 El ahorro de memoria en los años 60 y 70, La carrera contrarreloj para evitar el colapso 1

2.2 Registros del Senado de EE.UU. y el costo multimillonario 2

3. La nueva cuenta regresiva: hacia el 2038 2-3

3.1 Gangnam Style y el límite de YouTube (2,147,483,647) 2-3

3.2 El reloj que rebota a 1901 3

4. Los temas técnicos y cómo funciona 3-7

4.1 Unix Epoch y `time_t` de 32 bits 3-4

4.2 Fecha del desbordamiento: 19 de enero de 2038 4

4.3 Impacto en planificadores de procesos 4-6

4.4 Consecuencias en sincronización de procesos hijos 6

Conclusión: El veredicto final 7

5.1 Buenas noticias: los sistemas modernos de 64 bits ya están listos 6-7

5.2 El verdadero peligro: la desidia y los sistemas olvidados 7

Referencias 7

1. El Apocalipsis Digital y la Profecía del Y2K38

Imagina un escenario donde la infraestructura moderna del mundo entero simplemente colapsa de la nada. El dinero virtual se borra de los registros, los bancos electrónicos dejan de funcionar, generando pérdidas millonarias instantáneas. En los cielos y las vías, los medios de transporte como trenes y aviones se auto-apagan en pleno trayecto, mientras que las compañías de telecomunicaciones fallan en cadena, dejando a millones de usuarios sin internet y sin señal móvil. Esto no es el guion de una película de catástrofes; es la consecuencia exacta de un reloj invisible llegando a su límite físico: el "Y2K38".

Esta amenaza es tan aterradora que se convirtió en el centro de una de las mayores leyendas de viajes en el tiempo de la era moderna. A principios del año 2000, un supuesto viajero del tiempo llamado John Titor surgió en internet afirmando venir de un futuro devastado. ¿Su misión principal? Viajar en el tiempo hasta el año 1975 para recuperar una antigua computadora IBM 5100. Según la historia de Titor, esta computadora poseía una función oculta en su código, la cual era la capacidad de leer y traducir el código de los primeros sistemas informáticos de la historia, Era la única herramienta capaz de acceder a lo más profundo de esos sistemas olvidados para repararlos y evitar el colapso provocado por el Y2K38. Más allá de si fue un brillante engaño de internet o pura ciencia ficción, el mito de Titor resuena hasta hoy porque refleja un terror muy real: el miedo a que nuestra propia tecnología sea exactamente lo que provoque nuestro final.

2.1 ¿De dónde surge este apocalipsis programado?

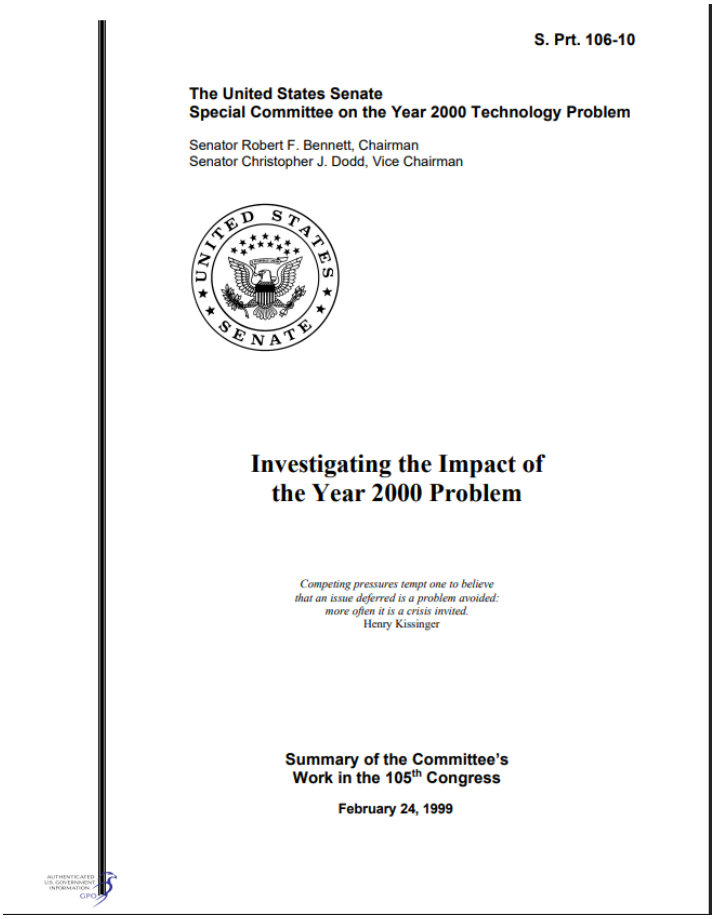
Para entender la amenaza que nos espera, primero debemos mirar atrás. En los años 60 y 70, la memoria de las computadoras era un recurso extremadamente costoso. Para ahorrar espacio, los programadores tomaron una decisión que parecía inofensiva: registrar los años usando solo sus últimos dos dígitos. El año 1998 era simplemente "98" como solemos escribir (26/04/75). Sin embargo, se dieron cuenta tarde de que cuando el calendario marcara el año 2000, los sistemas leerían "00" e interpretarían que habíamos regresado al año 1900. Esto podría generar grandes errores en todo tipo de programas, por ejemplo: Imagina que existía un programa encargado de calcular los intereses de un cliente basándose en la fecha. De pronto, al pasar de diciembre de 1999 a enero del 2000, un software mal diseñado podría interpretar que habían transcurrido menos de 100 años desde el último cobro. Como resultado, terminaría calculando intereses negativos, es decir, restando dinero en lugar de sumarlo. Se tuvo que invertir una fortuna a contrarreloj para actualizar los sistemas y evitar un colapso. El desastre logró evitarse, pero el precio fue una inversión multimillonaria de emergencia. Fue una carrera contrarreloj donde ingenieros de todo el mundo tuvieron que parchar infraestructuras clave justo antes de que el reloj marcara el cambio de milenio.

2.2 Registros del Comité Especial del Senado de EE. UU. (Problema del Año 2000)

El gobierno de los Estados Unidos formó un comité real dedicado exclusivamente a prevenir el desastre del Y2K. Los documentos desclasificados (en los dominios oficiales .gov) muestran testimonios sobre inyecciones de emergencia e impactos catastróficos si fallaban las telecomunicaciones, químicas y bancos.

Archivos oficiales del gobierno de EE.UU. (GovInfo.gov): El reporte "Investigating the Impact of the Y2K Problem" presentado ante el Senado advierte un gasto federal inicial superior a los 7,500 millones de dólares y proyecciones de impacto legal global de hasta 1 billón de dólares (1 trillón en inglés).

Enlace (Documento PDF Oficial del Senado):
https://www.govinfo.gov/content/pkg/GPO-CPRT-106sprt10/pdf/GPO-CPRT-106sprt10.pdf



S. Prt. 106-10

The United States Senate
Special Committee on the Year 2000 Technology Problem

Senator Robert F. Bennett, Chairman
Senator Christopher J. Dodd, Vice Chairman

Investigating the Impact of the Year 2000 Problem

Competing pressures tempt one to believe that an issue deferred is a problem avoided: more often it is a crisis invited.
Henry Kissinger

Summary of the Committee's Work in the 105th Congress

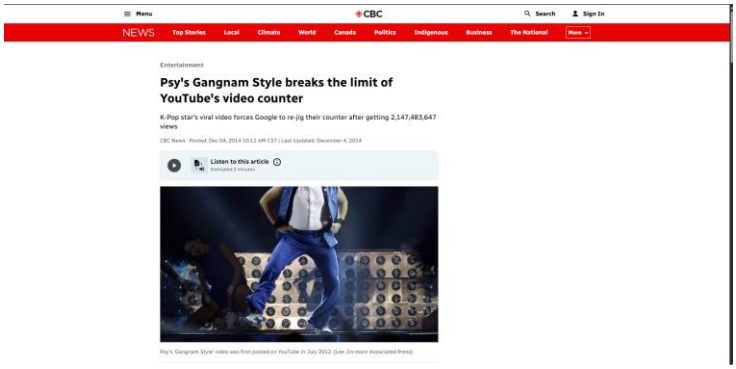
February 24, 1999

3. La cuenta regresiva hacia el 2038

3.1 Gangnam Style y el límite de YouTube (2,147,483,647) (¿Por qué tu teléfono no pasa del 31 de diciembre de 2037?)

Sobrevivimos al año 2000, pero estamos en una nueva cuenta regresiva. De manera curiosa, un aviso temprano de este nuevo límite nos lo dio el éxito musical Gangnam

Style en 2012, cuando su abrumadora popularidad rompió el contador de YouTube al superar los 2,147,483,647 de visualizaciones. Ese número no es una coincidencia. Es el valor máximo positivo que puede almacenar un número entero en una arquitectura de 32 bits. El verdadero peligro radica en que una inmensa cantidad de sistemas operativos, especialmente aquellos basados en UNIX, utilizan este sistema de 32 bits para medir el tiempo, contando los segundos transcurridos desde el 1 de enero de 1970. Si quieres hacer la prueba, puedes tomar tu teléfono e intentar adelantar la configuración del calendario al futuro. Notarás algo interesante: ya sea que uses Android o un iPhone, es imposible pasar del 31 de diciembre de 2037. El propio sistema operativo, ya sea por una limitación física o para proteger el código de sus aplicaciones antiguas, tiene prohibido por diseño acercarse a esa fecha límite.



https://www.cbc.ca/news/entertainment/psy-s-gangnam-style-breaks-the-limit-of-youtube-s-video-counter-1.2860186

3.2 El reloj que rebota a 1901

El reloj tiene un límite físico. A las 03:14:07 UTC del 19 de enero de 2038, el contador llegará a su tope. Si esto ocurre sin corrección, el sistema no sabrá qué hacer y el contador pasará a ser un número negativo, haciendo que los relojes internos reboten a 1901. Un fallo de esta magnitud afectaría inmediatamente las rutinas más críticas del núcleo de los sistemas operativos, colapsando tareas esenciales como la **planificación de procesos**, la sincronización de bases de datos y la administración de memoria.

TEMAS TECNICOS Y COMO FUNCIONA

4.1 ¿Qué es el Unix Epoch y time_t de 32 bits?

El Unix Epoch es el punto de partida del sistema de medición de tiempo en los sistemas operativos Unix y sus derivados (como Linux). Como señala wraycastle.com, "la época de Unix es un punto de referencia temporal que comienza a las 00:00:00 UTC del 1 de enero de 1970" (Wraycastle, 2024). Esta fecha fue elegida por su cercanía con la creación del sistema Unix en los laboratorios Bell, y por alinearse con el calendario gregoriano, eliminando así las complejidades de las zonas horarias.

A partir de ese momento, el Unix Timestamp (o marca de tiempo Unix) cuenta los segundos transcurridos de forma continua y lineal. Es decir, cada segundo que pasa aumenta el contador en uno. Este sistema es muy eficiente para las computadoras porque representa el tiempo como un simple número entero, evitando los problemas de formatos de fecha variables, años bisiestos o cambios de horario de verano.

El tiempo en estos sistemas se maneja a través de un tipo de dato definido en C llamado time_t. Es crucial destacar que time_t no es una simple variable que podamos modificar a nuestro antojo, sino un tipo de dato histórico que forma parte de la ABI (Application Binary Interface) del sistema operativo. En los sistemas Unix antiguos y arquitecturas de 32 bits, este tipo de dato se definió rígidamente como un entero de 32 bits con signo, lo que significa que puede representar números enteros desde -2,147,483,648 hasta 2,147,483,647. El valor máximo positivo (2,147,483,647) es el techo que el sistema puede contar antes de desbordarse. Dado que el tamaño de time_t está profundamente arraigado en la ABI, no se puede cambiar arbitrariamente a 64 bits simplemente porque lo decidamos; intentar hacerlo rompería la compatibilidad binaria de todo el software, bibliotecas y aplicaciones que ya fueron compiladas asumiendo que el tiempo medía 32 bits.

4.2 Fecha del desbordamiento: 19 de enero de 2038

El límite de 2,147,483,647 segundos se alcanzará en una fecha y hora muy concretas: el 19 de enero de 2038 a las 03:14:07 UTC. En ese preciso instante, el contador de tiempo Unix habrá registrado exactamente ese número de segundos desde el 1 de enero de 1970.

¿Qué ocurre un segundo después, a las 03:14:08 UTC? Como explica *utilitybro.com*, "cuando el contador de segundos Unix llegue a ese número, los sistemas de 32 bits no podrán contar más. El contador se desbordará y saltará al número negativo más bajo: -2,147,483,648, que corresponde al año 1901" (UtilityBro, 2025). Es decir, el sistema no avanzará al siguiente número positivo (2,147,483,648) porque en 32 bits con signo ese valor no existe. En su lugar, el contador "da la vuelta" y se convierte en el número negativo más bajo. La razón exacta de este 'viaje en el tiempo' a 1901 radica en la equivalencia de esos segundos. El número máximo representable por un entero con signo de 32 bits es 2,147,483,647 segundos, lo cual equivale a poco más de 68 años. Si sumamos 68 años a partir del Unix Epoch (1 de enero de 1970), aterrizamos precisamente en enero de 2038. Al ocurrir el desbordamiento, el sistema salta al extremo negativo (-2,147,483,648 segundos). Como el sistema evalúa el tiempo contando los segundos desde 1970, tener un valor negativo significa retroceder en el tiempo. Si retrocedemos exactamente esos mismos 68 años desde el 1 de enero de 1970, el calendario matemático del sistema nos arroja al 13 de diciembre de 1901.

Para entenderlo mejor: es como un odómetro de un auto que marca 999,999 kilómetros. Al recorrer un kilómetro más, no marca 1,000,000 sino que vuelve a 000,000. En este caso, el odómetro no vuelve a cero, sino que salta a un número negativo que el sistema interpreta como el año 1901, es decir, 137 años antes del Epoch.

4.3 Impacto en Cálculo de tiempos en planificación (schedulers)

El planificador de procesos (scheduler) es el encargado de decidir qué proceso se ejecuta y durante cuánto tiempo. Para esto, utiliza algoritmos que asignan a cada proceso un quantum o intervalo de tiempo de CPU. En sistemas que usan el algoritmo Round-Robin, cada proceso recibe un turno de duración fija. La función del sistema que devuelve este valor es sched_rr_get_interval().

En sistemas de 32 bits, esta función utiliza time_t de 32 bits para representar ese intervalo de tiempo. Cuando ocurra el desbordamiento del año 2038, los valores devueltos por sched_rr_get_interval() podrían volverse negativos o absurdamente grandes. Esto significa que el planificador podría asignar a un proceso tiempos de CPU negativos (imposible de ejecutar) o intervalos enormes que acaparen el procesador indefinidamente.

Además, el planificador necesita calcular constantemente cuánto tiempo ha usado ya un proceso y cuánto le queda. Si el sistema de tiempo base está roto por el desbordamiento, esas cuentas serán incorrectas, provocando que algunos procesos se ejecuten muy poco y otros demasiado, rompiendo la equidad y el rendimiento esperado del sistema.

Desbordamiento negativo y colas de espera rotas

Cuando el contador de 32 bits se desborda, no solo se detiene, sino que salta a un número negativo enorme (-2,147,483,648 segundos), que el sistema interpreta como el 13 de diciembre de 1901. Este "viaje al pasado" rompe por completo las colas de espera del sistema operativo.

Las colas de espera son estructuras fundamentales donde el sistema pone a los procesos que están temporalmente detenidos. Por ejemplo, cuando un proceso necesita esperar unos segundos antes de continuar, el planificador lo coloca en una cola de espera y programa su despertar para un momento específico en el futuro. El problema es que este cálculo depende de comparar la hora actual con la hora de despertar.

Cuando la hora actual se vuelve negativa por el desbordamiento, toda esta lógica falla. Como documentaron los desarrolladores del kernel de Linux al realizar los parches Y2038, "las aplicaciones intentan establecer temporizadores después de que time_t se vuelve negativo y entran en un bucle infinito" (John Stultz, lista de correo del kernel). Es decir, los procesos se quedan atrapados para siempre, consumiendo memoria y recursos sin liberarse, porque el sistema rechaza los tiempos negativos y devuelve errores una y otra vez.

El resultado práctico es que los procesos bloqueados nunca despiertan, las colas de espera se llenan de procesos zombies y el planificador pierde la capacidad de gestionar correctamente los tiempos de ejecución

Fallo en algoritmos de prioridad por tiempo

Algunos planificadores de procesos, como el Completely Fair Scheduler (CFS) de Linux, dependen del tiempo para decidir qué proceso tiene prioridad sobre otro . Estos algoritmos calculan constantemente cuánto tiempo de CPU ha recibido cada proceso y comparan esos valores para determinar quién debe ejecutarse a continuación.

Cuando el contador de tiempo se desborda y salta a valores negativos, estos cálculos pierden todo sentido. Un proceso que apenas ha usado tiempo de CPU podría aparecer como si hubiera consumido una cantidad negativa o enormemente grande, lo que confunde al planificador y le hace tomar decisiones incorrectas.

Esto afecta a las llamadas al sistema, pero también a comandos específicos y a otras interfaces . Entre las funciones más relevantes para la planificación que requirieron cambios se encuentra sched_rr_get_interval(), que devuelve el intervalo de tiempo asignado a cada proceso en el algoritmo Round-Robin .

El resultado práctico es que el planificador deja de ser "justo": algunos procesos reciben mucho más tiempo de CPU del que deberían, mientras que otros prácticamente no se ejecutan. En sistemas de tiempo real, donde los plazos de entrega son críticos, esto puede provocar fallos catastróficos.

4.4 **Consecuencias en sincronización de procesos hijos (Llamadas afectadas: waitpid(), wait(), nanosleep())**

Cuando un proceso padre necesita esperar a que sus procesos hijos terminen o cambien de estado, utiliza funciones como wait() y waitpid(). Estas funciones son esenciales para la sincronización entre procesos padre e hijo, un tema central que estamos viendo en clase. El problema es que estas funciones dependen internamente de temporizadores y sistemas de medición de tiempo que, en arquitecturas de 32 bits, se desbordarán en 2038.

Tomemos como ejemplo nanosleep(). Esta función se usa para pausar un proceso durante una cantidad específica de segundos o nanosegundos. Podría ser usada por un proceso hijo para esperar antes de enviar una señal a su padre, o por un proceso padre para dar tiempo a que su hijo realice una tarea. Con el desbordamiento del año 2038, el kernel podría interpretar mal el tiempo: si le pides "espera 5 segundos", el sistema podría leer un número negativo enorme y creer que tiene que esperar más de cien años en el pasado, causando que el proceso se quede congelado para siempre o que despierte inmediatamente de forma incorrecta.

Otro ejemplo práctico es waitpid(), que permite esperar a un proceso hijo con un tiempo límite. Imagina un servidor web que crea procesos hijos para atender clientes. El servidor padre usa waitpid() para esperar que un hijo termine, pero solo está dispuesto a esperar 10 segundos antes de matarlo y continuar. Si el contador de tiempo se desborda justo en ese momento, el servidor podría interpretar que ya pasaron 136 años desde que empezó a esperar, abortando el proceso hijo inmediatamente aunque estuviera a punto de terminar. O peor aún, podría calcular una fecha de expiración en el pasado remoto y quedar atrapado en un bucle infinito esperando algo que nunca ocurrirá.

La solución que han implementado los desarrolladores del kernel es reescribir funciones como nanosleep para que usen operaciones basadas en futex con tiempos de 64 bits, evitando así el desbordamiento. Sin embargo, mientras no se actualicen todos los sistemas, los procesos que dependan de estas llamadas de 32 bits quedarán vulnerables.

Conclusión

El Veredicto: ¿Un nuevo fin del mundo o un fallo evitable?

5.1 Buenas noticias: El esfuerzo titánico del software libre y los sistemas de 64 bits

Después de imaginar el fin del mundo con el problema del 2038, tenemos buenas noticias: la realidad es mucho más manejable. La gran ventaja frente a crisis pasadas es que la comunidad tecnológica vio venir este colapso con décadas de anticipación y tomó cartas en el asunto.

Este nivel de preparación no ocurrió por arte de magia, sino gracias al esfuerzo monumental de la comunidad de código abierto. Un momento clave que encendió las alarmas y concientizó a la industria ocurrió en 2017, durante el congreso DebConf17 en Montreal. Allí, el reconocido desarrollador de Debian, Steve McIntyre, presentó la histórica charla "It's The End of the World... (in 21 Years)".

McIntyre expuso la cruda realidad del Y2K38: no bastaba con cruzar los dedos y esperar a que todo el mundo comprara procesadores de 64 bits para el año 2038. El verdadero reto de ingeniería era modificar el núcleo de los sistemas operativos para que las arquitecturas de 32 bits pudieran sobrevivir, lo que implicaba el gigantesco trabajo de migrar el tipo de dato time_t a 64 bits sin destruir la compatibilidad binaria (ABI) del software existente.

Gracias a advertencias como la de McIntyre, la comunidad se puso a trabajar. Proyectos críticos como el Kernel de Linux comenzaron a rediseñar sus entrañas, logrando (a partir de la versión 5.6) que incluso los sistemas de 32 bits pudieran manejar el tiempo de forma segura. A la par de estas soluciones de software, la evolución natural del hardware ha empujado a que la inmensa mayoría de computadoras, servidores y teléfonos modernos operen nativamente en 64 bits. En estas arquitecturas, el límite para contar segundos se extiende por miles de millones de años.

Es decir, gracias al trabajo preventivo de los ingenieros de software y la actualización del hardware, para los equipos modernos este problema simplemente ya no existe. El apocalipsis digital del que hablaba la leyenda de John Titor está oficialmente cancelado para nosotros como usuarios comunes.

5.2 El verdadero peligro: la desidia y los sistemas olvidados

¿Dónde está el verdadero peligro entonces? En la desidia corporativa e institucional. El riesgo real se esconde en los "sistemas heredados" (legacy systems) que nadie ha querido actualizar: infraestructuras de redes eléctricas, sistemas militares, dispositivos embebidos (IoT), antenas de telecomunicaciones o cajeros automáticos que operan en el olvido con tecnología antigua.

Como explicamos anteriormente con la inmutabilidad de la ABI (Application Binary Interface), rescatar estos equipos no es tan simple como enviarles un parche o una actualización de software por internet. El software propietario y crítico que corre en estas máquinas fue compilado hace décadas asumiendo rígidamente un tipo de dato time_t de 32 bits. Intentar forzar el sistema operativo de esos equipos para que cuente el tiempo en 64 bits rompería la compatibilidad binaria de inmediato, provocando que los programas base colapsen al instante.

Para salvar verdaderamente estas infraestructuras, las organizaciones tienen que localizar código fuente muy antiguio o que muchas veces ya se perdió, para recompilarlo desde cero, o derechamente reemplazar el hardware físico y los microcontroladores de 32 bits por equipos nuevos. Esto representa una inversión y un esfuerzo de ingeniería multimillonario.

Si las empresas deciden ahorrarse este dinero, escudándose en la excusa de que "el 2038 todavía está muy lejos", esos sistemas colapsarán inevitablemente al llegar al límite de la variable, causando apagones, fallos logísticos y pérdidas millonarias en los sectores donde menos lo esperamos.

Pero, si hacemos a un lado estas bombas de tiempo industriales, la realidad es que gracias al esfuerzo de las comunidades de desarrollo y la evolución de los equipos actuales, para la mayor parte del mundo tecnológico moderno el 19 de enero de 2038 será simplemente un martes cualquiera en el calendario.

REFERENCIAS

- CBC News. (2014, diciembre 3). *Psy's Gangnam Style breaks the limit of YouTube's video counter*. Recuperado de https://www.cbc.ca/news/entertainment/psy-s-gangnam-style-breaks-the-limit-of-youtube-s-video-counter-1.2860186
- Kaspersky. (2026). *Epochalypse Now, or how to address the Y2K38 problem in organizations*. Recuperado de https://latam.kaspersky.com/blog/mitigating-y2k38-vulnerability-in-organization/28914/
- U.S. Government Publishing Office. (2000). *Investigating the impact of the Y2K problem* (GPO-CPRT-106sprt10). Recuperado de https://www.govinfo.gov/content/pkg/GPO-CPRT-106sprt10/pdf/GPO-CPRT-106sprt10.pdf

- UtilityBro. (2025). *Qué es el Tiempo Unix (Epoch Time) y por qué es crítico en 2025*. Recuperado de https://utilitybro.com/blog/que-es-unix-timestamp.html
- Wraycastle. (2024). *Comprensión de la marca de tiempo de Unix Epoch: una guía práctica para todos*. Recuperado de https://wraycastle.com/es/blogs/knowledge-base/unix-epoch-timestamp
- Kamp, P.-H. (2022). It's About Time. Communications of the ACM. Recuperado de https://cacm.acm.org/opinion/its-about-time/
- Corbet, J. (2017). *The end of time (for 32-bit systems)*. LWN.net. Recuperado de https://lwn.net/Articles/732794/
- McIntyre, S. (2017). *It's The End of the World... (in 21 Years)*. Charla presentada en DebConf17, Montreal. Recuperado de https://debconf17.debconf.org/talks/31/
- Internet Archive. (2006). *The Complete John Titor posts*. Recuperado de https://archive.org/stream/completetitor/CompleteTitor_djvu.txt